College Expense Tracker - Project Report

1. Abstract

Managing finances effectively is a common challenge for college students, who often struggle to track daily expenses like food, transport, and books within a fixed monthly budget. The **College Expense Tracker** is a Java-based console application designed to help students record, monitor, and analyze their spending habits seamlessly. By applying fundamental Object-Oriented Programming (OOP) concepts such as Abstraction, Encapsulation, Inheritance, and Polymorphism, the application provides a modular, robust, and extensible system. It enables students to calculate remaining budgets, visualize spending categories, and set actionable savings goals. This project emphasizes backend architecture and foundational software engineering principles while delivering a highly practical money management tool.

2. Functional Requirement

The application satisfies the following core functional requirements:

1. **User Profiling:** Users can create a personal profile by providing their Full Name, Roll Number, and an initial Monthly Budget.
2. **Expense Tracking:** Users can add new expenses, logging the specific amount, description, and categorizing them into predefined types (e.g., Food, Transport, Books, Entertainment).
3. **Recurring Expenses:** The backend accommodates recurring expense models that automatically subtract set amounts during monthly cycles.
4. **Budget Monitoring:** A safety check is triggered to warn users if a newly added expense exceeds their current remaining budget.
5. **Category Analytics:** The application calculates and aggregates expenses by category, determining the top-spending category.
6. **Savings Goals:** Users can establish financial goals (e.g., saving money for a new device) and track their progress based on any leftover budget.
7. **Detailed Summaries:** The system can generate both individual expense histories and comprehensive monthly summaries.

3. UML Diagrams

3.1 Use Case Diagram

3.2 Activity Diagram

Syntax error in text
mermaid version 11.14.0

3.3 Class Diagram

Syntax error in text
mermaid version 11.14.0

4. Structure or Flow Diagram

Syntax error in text
mermaid version 11.14.0

5. Git Hub Link

(Please insert your GitHub repository link here)
`https://github.com/YourUsername/CollegeExpenseTracker`

6. Input: Put all the OOP Concepts in code

Below is the direct application of fundamental OOP concepts inside the College Expense Tracker:

- **Abstraction**: Applied by creating `public abstract class User`. The `User` class acts as a template defining core properties, while enforcing children to implement details via the abstract method `public abstract void displayInfo();`.

```
public abstract class User {
    protected String name;
    protected double monthlyBudget;

    // Abstract method forces subclasses to define their own display logic
```

```
    public abstract void displayInfo();
}
```

- **Encapsulation**: Object state is hidden to prevent unauthorized modifications. Variables like `private String rollNo;` and `private ArrayList<Expense> expenses;` in `Student`, as well as `private double amount;` in `Expense` are kept private or protected, accessible only through appropriate getters/setters.

```
public class Expense {
    private double amount;
    private Category category;
    private String description;

    public double getAmount() {
        return amount;
    }
    // ... other getters
}
```

- **Inheritance**: The hierarchy builds logically using `extends`. The `Student` class represents an "IS-A" relation with `User` (`public class Student extends User`), inheriting the `name` and `monthlyBudget` characteristics. Similarly, `RecurringExpense extends Expense` to handle repetitive expenses while reusing standard properties.

```
public class Student extends User {
    private String rollNo;

    public Student(String name, double monthlyBudget, String rollNo) {
        super(name, monthlyBudget); // Calls parent constructor in User
        this.rollNo = rollNo;
    }
}
```

- **Polymorphism**:
    - *Method Overriding*: The `Student` class overrides the parent's abstract method (`@Override public void displayInfo()`). `Expense` and `RecurringExpense` effectively override `toString()` to output formatted descriptive details.

```
@Override
public void displayInfo() {
    System.out.println("Name    : " + name);
    System.out.println("Remaining: Rs " + getRemainingBudget());
    // Custom implementation entirely different from another subclass
}
```

- **Composition & Aggregation**: The relationships between objects are modeled effectively.
  - *Composition (Strong)*: Standard `Expense` and `RecurringExpense` objects are created directly within the `Student` functionality, bound to their lifecycle.
  - *Aggregation (Weak "HAS-A")*: A `Student` possesses an optional `SavingsGoal`, demonstrating a flexible component relationship.

```java
public class Student {
    // Composition: Expenses only exist relative to the student creating them
    private ArrayList<Expense> expenses = new ArrayList<>();

    // Aggregation: A goal can be independently conceived and assigned later
    private SavingsGoal goal;
}
```

- **Collections Framework**: Dynamic storage limits run-time restrictions. Uses `ArrayList<Expense>` to maintain variable-length expense histories and `HashMap<Category, Double>` for highly efficient mapping and retrieval of category-wise spending.

```java
// ArrayList for contiguous data storage of generic variable length
private ArrayList<Expense> expenses;

// HashMap linking Enum Categories to Double amounts for O(1) mathematical
lookups
private HashMap<Category, Double> categoryTotals = new HashMap<>();
```

7. Output: Console Executions

(Below are functional terminal snapshots of the executed Java Code showcasing the program's operations)

Fig 1: Application Startup and Profile Creation

```
╔═══════════════════════════╗
║   COLLEGE EXPENSE TRACKER    ║
╚═══════════════════════════╝

--- Setup Profile ---
Full Name   : Ansh
Roll Number : 24XYZ
Monthly Budget (Rs): 5000
✓ Profile created successfully! Welcome, Ansh!
```

Fig 2: Adding Expenses and Budget Safety Checking

```
======= MENU [Ansh] =======
1. Add Expense
2. View All Expenses
3. View Budget Status
4. View Category Totals
5. Manage Recurring Expenses
6. Manage Savings Goal
7. Monthly Summary
8. Exit
================================
Choose: 1

--- ADD EXPENSE ---
1=FOOD  2=TRANSPORT  3=BOOKS  4=ENTERTAINMENT
Category: 1
Amount (Rs): 250
Description: Lunch
✓ Added! Remaining Budget: Rs 4750.0

... (Adding another large expense) ...

--- ADD EXPENSE ---
1=FOOD  2=TRANSPORT  3=BOOKS  4=ENTERTAINMENT
Category: 2
Amount (Rs): 6000
Description: Flight Tickets
⚠ WARNING: This exceeds your remaining budget!
Still add? (y/n): y
✓ Added! Remaining Budget: Rs -1250.0
```

Fig 3: Viewing Budget Status and Category Totals

```
======= MENU [Ansh] =======
...
Choose: 3

--- Budget Status ---
Name    : Ansh
Roll No : 24XYZ
Budget  : Rs 5000.0
Spent   : Rs 6250.0
Remaining: Rs -1250.0

======= MENU [Ansh] =======
...
Choose: 4

--- Category Totals ---
```

```
TRANSPORT : Rs 6000.0
FOOD : Rs 250.0
```

Fig 4: Advanced Features (Recurring Expenses, Goals, and Summaries)

```
======= MENU [Ansh] =======
...
Choose: 5

--- RECURRING EXPENSES ---
1. Add New Recurring Expense
2. Apply Pending Recurring Expenses (Monthly Charge)
3. Back
Choose: 1
1=FOOD  2=TRANSPORT  3=BOOKS  4=ENTERTAINMENT
Category: 4
Amount (Rs): 300
Charge Day of Month (1-31): 15
Description: Spotify
✓ Recurring Expense template created! It will automatically charge on day 15

======= MENU [Ansh] =======
...
Choose: 5

--- RECURRING EXPENSES ---
1. Add New Recurring Expense
2. Apply Pending Recurring Expenses (Monthly Charge)
3. Back
Choose: 2

--- Applying Recurring Expenses for New Month ---
Applying: Spotify (Rs 300.0)
✓ All recurring expenses have been charged. Remaining Budget: Rs -1550.0

======= MENU [Ansh] =======
...
Choose: 6

--- MANAGE SAVINGS GOAL ---
New Goal Name: New Phone
Target Amount (Rs): 15000
✓ Goal Created: New Phone (Target: Rs 15000.0)

======= MENU [Ansh] =======
...
Choose: 6

--- MANAGE SAVINGS GOAL ---
Goal     : New Phone
Target   : Rs 15000.0
Saved    : Rs 0.0
```

```
Progress : 0.0%
Remaining: Rs 15000.0
Progress : [....................]

1. Add money to existing goal
2. Set a new goal (overwrite)
3. Back to menu
Choose: 1
Amount to add (Rs): 2000
✓ Added Rs 2000.0 to savings!

======= MENU [Ansh] =======
...
Choose: 7

========= MONTHLY SUMMARY =========
Budget     : Rs 5000.0
Spent      : Rs 6550.0
Saved      : Rs -1550.0 (-31.0%)
Top Category: TRANSPORT (Rs 6000.0)
Goal Progress: 13.3%
====================================
```

8. Conclusion

The College Expense Tracker effectively solves the challenge of budget management for students while simultaneously demonstrating structured object-oriented design parameters. By adhering correctly to classes, inheritance boundaries, and data encapsulation, the resulting program eliminates common procedural programming flaws. Its simple structure makes it lightweight, while utilizing Java Collections constructs a robust base prepared for dynamic data tracking without hardcoded size limits. This project proves how foundational software engineering principles resolve everyday problems analytically.

9. Future Enhancement

While currently operational, several areas could broaden the project's capabilities:

1. **Database Integration:** Upgrading from local, memory-based Array-Lists to persistent SQL or NoSQL databases to retain financial histories across program restarts.
2. **Graphical User Interface (GUI) & Web Frontend:** Shifting from CLI to an interactive interface utilizing JS Frameworks interacting with Java via RESTful APIs.
3. **Authentication:** Adding secure User login protocols, allowing the same application to safely separate profiles for multiple users securely.
4. **Data Visualization Export:** Introducing external library functionality to export visually engaging charts (like pie charts for categories) natively to PDF or image files.